

LP V Assignment No 3

Name : Bhure Sangmeshwar Dattatray

Roll No : 07

```
In [5]: # =====  
# CNN Classifier – Fashion MNIST (Simplified & Stable)  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
warnings.filterwarnings("ignore")  
  
import tensorflow as tf  
from tensorflow import keras  
from tensorflow.keras import layers  
from tensorflow.keras.utils import to_categorical  
  
from sklearn.metrics import classification_report, confusion_matrix  
  
print("TensorFlow:", tf.__version__)  
  
# -----  
# CLASS NAMES  
# -----  
CLASS_NAMES = [  
    "T-shirt", "Trouser", "Pullover", "Dress", "Coat",  
    "Sandal", "Shirt", "Sneaker", "Bag", "Boot"  
]  
  
# -----  
# LOAD DATA  
# -----  
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()  
  
# Normalize  
x_train = x_train / 255.0  
x_test = x_test / 255.0  
  
# Reshape  
x_train = x_train.reshape(-1, 28, 28, 1)  
x_test = x_test.reshape(-1, 28, 28, 1)  
  
# One-hot encoding  
y_train_cat = to_categorical(y_train)  
y_test_cat = to_categorical(y_test)  
  
print("Data loaded successfully")  
  
# -----  
# BUILD MODEL
```

```

# -----
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),

    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# -----
# TRAIN MODEL
# -----
history = model.fit(
    x_train, y_train_cat,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)

# -----
# EVALUATION
# -----
loss, acc = model.evaluate(x_test, y_test_cat)
print("\nTest Accuracy:", acc)

# Predictions
y_pred = np.argmax(model.predict(x_test), axis=1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# -----
# CONFUSION MATRIX
# -----
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")

```

```

plt.ylabel("Actual")
plt.show()

# -----
# SAMPLE PREDICTIONS
# -----
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(CLASS_NAMES[y_pred[i]])
    plt.axis('off')

plt.show()

# -----
# SAVE MODEL
# -----
model.save("fashion_cnn_model.keras")

print("\n✅ Practical Completed Successfully")

```

TensorFlow: 2.21.0

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>

29515/29515 ————— 0s 2us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>

26421880/26421880 ————— 4s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>

5148/5148 ————— 0s 1us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>

4422102/4422102 ————— 1s 0us/step

Data loaded successfully

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 10)	

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

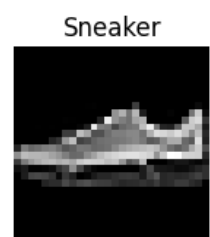
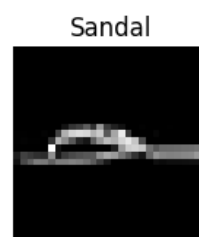
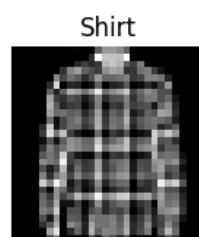
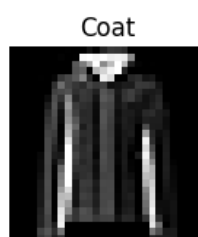
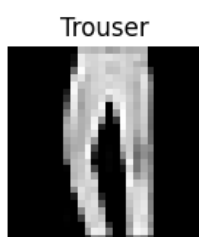
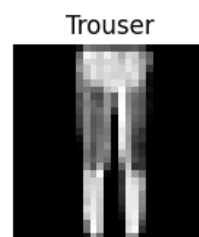
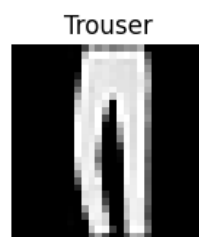
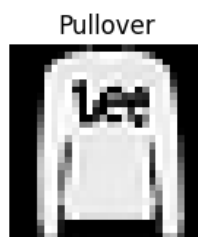
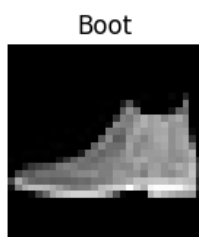
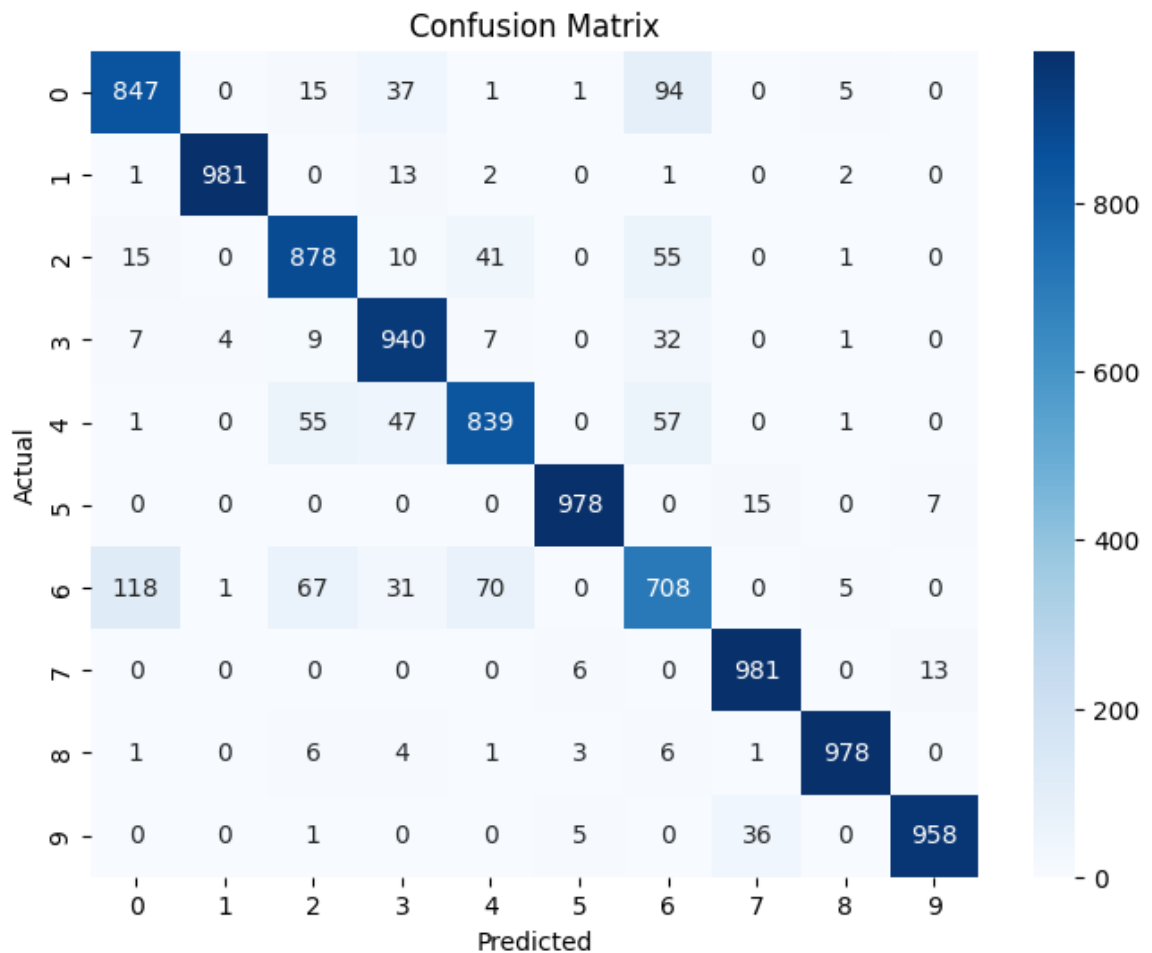
Non-trainable params: 0 (0.00 B)

Epoch 1/10
750/750  **8s** 9ms/step - accuracy: 0.7719 - loss: 0.6282 - val_accuracy: 0.8527 - val_loss: 0.4069
Epoch 2/10
750/750  **6s** 8ms/step - accuracy: 0.8518 - loss: 0.4133 - val_accuracy: 0.8776 - val_loss: 0.3307
Epoch 3/10
750/750  **6s** 8ms/step - accuracy: 0.8724 - loss: 0.3579 - val_accuracy: 0.8917 - val_loss: 0.3032
Epoch 4/10
750/750  **6s** 8ms/step - accuracy: 0.8842 - loss: 0.3202 - val_accuracy: 0.8967 - val_loss: 0.2802
Epoch 5/10
750/750  **6s** 8ms/step - accuracy: 0.8916 - loss: 0.2983 - val_accuracy: 0.9013 - val_loss: 0.2666
Epoch 6/10
750/750  **6s** 9ms/step - accuracy: 0.8992 - loss: 0.2760 - val_accuracy: 0.9051 - val_loss: 0.2572
Epoch 7/10
750/750  **7s** 9ms/step - accuracy: 0.9056 - loss: 0.2588 - val_accuracy: 0.9057 - val_loss: 0.2583
Epoch 8/10
750/750  **7s** 9ms/step - accuracy: 0.9108 - loss: 0.2430 - val_accuracy: 0.9091 - val_loss: 0.2444
Epoch 9/10
750/750  **7s** 9ms/step - accuracy: 0.9165 - loss: 0.2289 - val_accuracy: 0.9120 - val_loss: 0.2470
Epoch 10/10
750/750  **7s** 9ms/step - accuracy: 0.9181 - loss: 0.2237 - val_accuracy: 0.9157 - val_loss: 0.2408
313/313  **0s** 2ms/step - accuracy: 0.9088 - loss: 0.2542

Test Accuracy: 0.9088000059127808
313/313  **1s** 2ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.85	0.85	1000
1	0.99	0.98	0.99	1000
2	0.85	0.88	0.86	1000
3	0.87	0.94	0.90	1000
4	0.87	0.84	0.86	1000
5	0.98	0.98	0.98	1000
6	0.74	0.71	0.73	1000
7	0.95	0.98	0.97	1000
8	0.98	0.98	0.98	1000
9	0.98	0.96	0.97	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000



✅ Practical Completed Successfully

In []: